Truthiness

While Ruby has literal true and false values, it allows *any* object to be used in a conditional. The rules are very simple: false and nil are both treated as false, and everything else is treated as true (even the number zero!).

In a nod to comedian Stephen Colbert, Rubyists refer to these categories of values as *truthy* and *falsey* (or *falsy*; there's no standard spelling).[1] RSpec follows this lead with its truthiness matchers:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb

```
expect(true).to be_truthy
expect(0).to be_truthy
expect(false).not_to be_truthy
expect(nil).not_to be_truthy

# ...and on the flip side:
expect(false).to be_falsey
expect(nil).to be_falsey
expect(true).not_to be_falsey
expect(0).not_to be_falsey
```

If you find the language of "be truthy" and "be falsey" distasteful, bear in mind that the naming is intentional. It's a subtle nudge that you've chosen a more informal matcher. If you want to specify that a value is *precisely* equal to true or false, simply use one of the equality matchers we described in the last section:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb

```
expect(1.odd?).to be true
expect(2.odd?).to eq false
```

Like the equality matchers we saw earlier, truthiness matchers have aliases designed to read well in composed matcher expressions:

- be_truthy is aliased as a_truthy_value.
- be_falsey is aliased as be_falsy, a_falsey_value and a_falsy_value.

Operator Comparisons

We've used the be method with arguments before, as in expect(answer).to be(42). This method has another form, one without arguments. With it, you can perform greater-than and less-than comparisons (or use any Ruby binary operator):

1. http://www.cc.com/video-clips/63ite2/the-colbert-report-the-word---truthiness